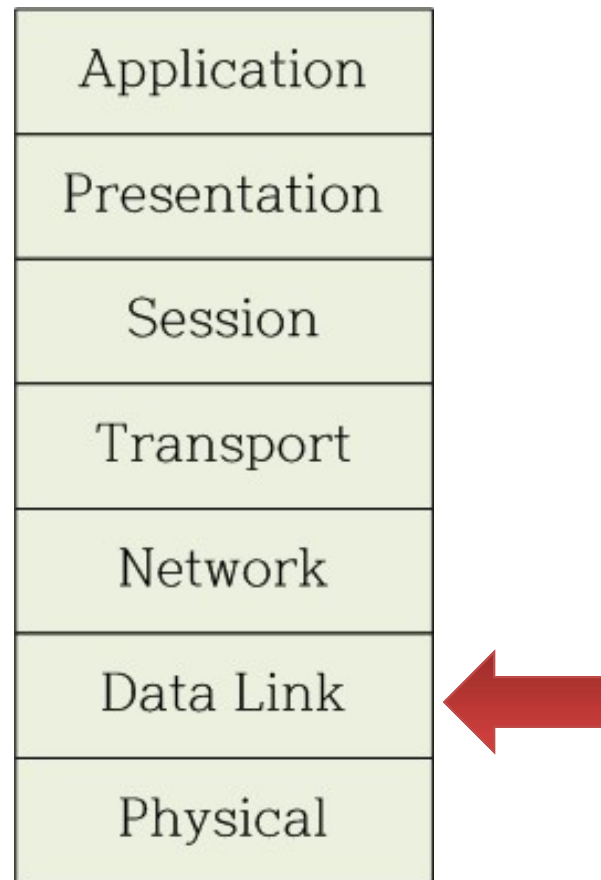


Error Detection and Correction

Data Link Layer

- OSI 7 layer model



※ Error detection is also used in other layers (e.g., IP checksum, TCP checksum)

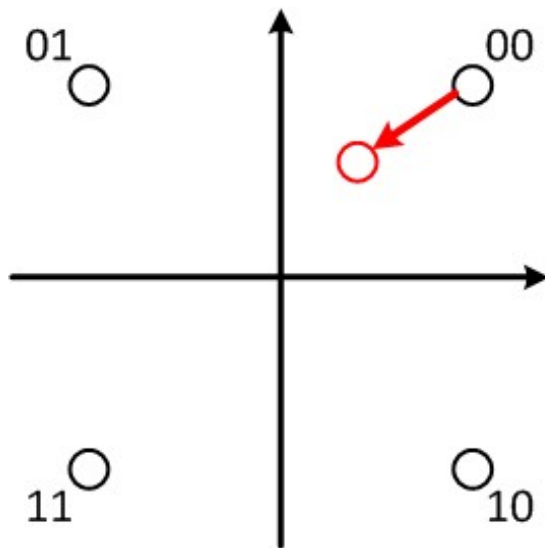
Errors

What is an error?

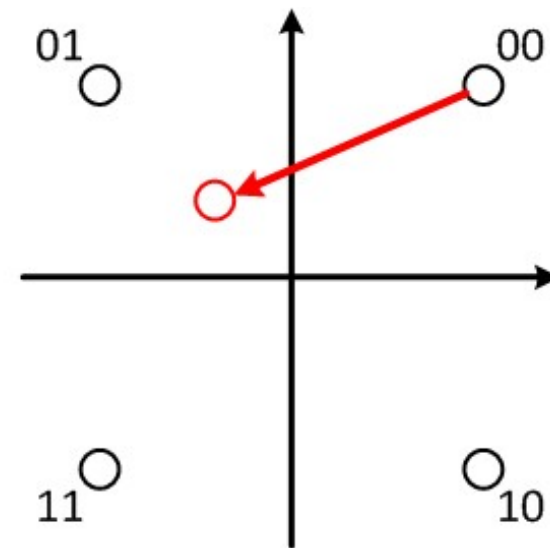
- In data link layer, data is a sequence of bits
- An error occurs when a bit is flipped during transmission
 - $0 \rightarrow 1$
 - $1 \rightarrow 0$
- The number of flipped bits in a block determines the severity of the error

Why do errors occur?

- attenuation, distortion, noise, interference
- amplitude, frequency and phase of signal changes



no error



error

n-bit error

- Suppose sender sends a block of data "01101100".
- Suppose the receiver receives "01101101".
- Then we say a 1-bit error occurred.

- If the receiver receives "0101101" → 3-bit error

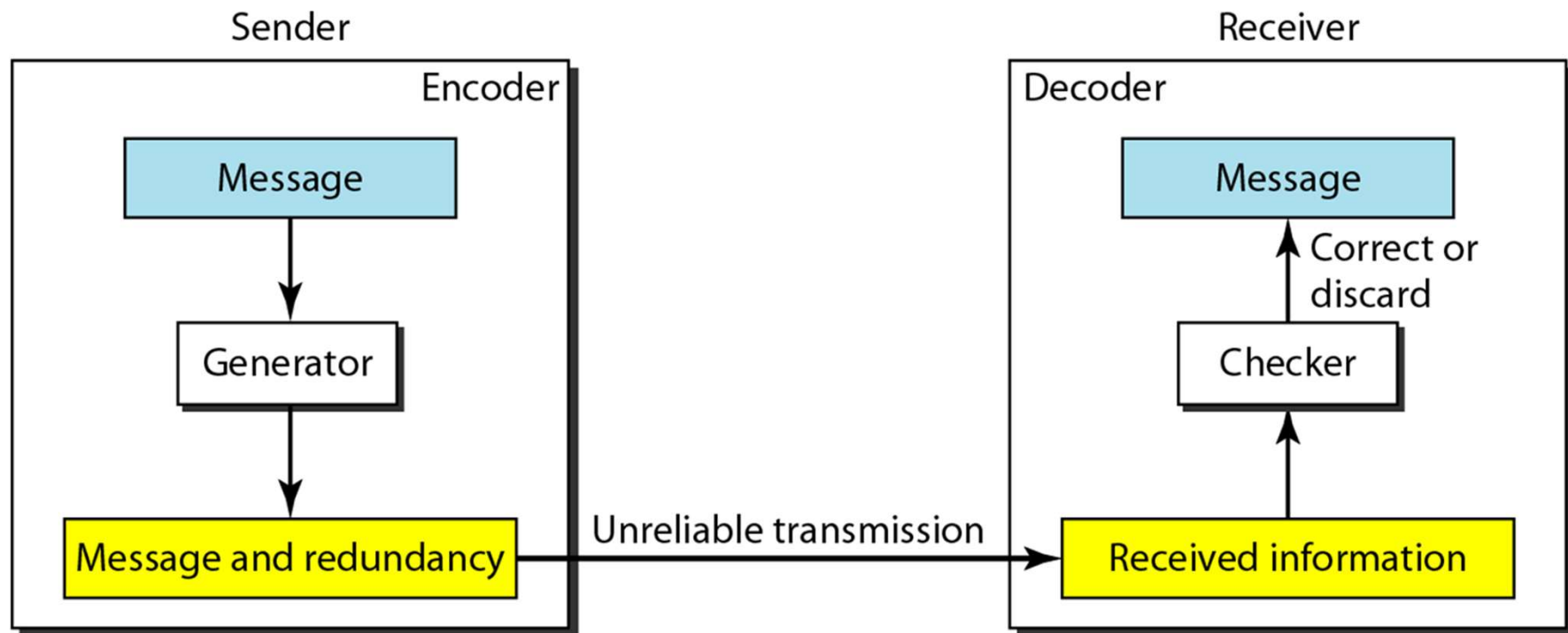
- The unit of error detection/correction depends on the block size

Error detection vs. correction

- Error detection
 - Know that an error has occurred
 - Don't know how to correct
- Error correction
 - Know that an error has occurred
 - Can correct the error

How error detection works

- How can we know that an error has occurred?
 - Add **redundancy** (extra bits) to the original message



Block Coding for Error Detection/Correction

Block coding

- Block coding was used in physical layer to avoid continuous zeros (e.g., 4B/5B)
- Block coding is also used in data link layer for error detection/correction
- Same idea: add redundant bits to the original data

Block coding

- Terms
 - dataword: a block of original message
 - codeword: block-coded message
- Generate an n-bit codeword from a k-bit dataword
 - dataword: k bits
 - codeword: n bits
 - r: redundancy
 - $r = n - k$

Block coding



2^k Datawords, each of k bits



2^n Codewords, each of n bits (only 2^k of them are valid)

Error detection using block coding

- We can block-code the message as the table below
- What is k ?
- What is n ?
- What is r ?
- What do we actually transmit if we want to send "01101100"?

<i>Datawords</i>	<i>Codewords</i>
00	000
01	011
10	101
11	110

Error detection using block coding

- The sender wants to send "01".
- The sender will send "011" using block coding.
- Suppose receiver received "011" (no error).
- What should the receiver do?

<i>Datawords</i>	<i>Codewords</i>
00	000
01	011
10	101
11	110

Error detection using block coding

- The sender wants to send "01".
- The sender will send "011" using block coding.
- Suppose receiver received "111" (1-bit error).
- What should the receiver do?

<i>Datawords</i>	<i>Codewords</i>
00	000
01	011
10	101
11	110

Error detection using block coding

- The sender wants to send "01".
- The sender will send "011" using block coding.
- Suppose receiver received "000" (2-bit error).
- What should the receiver do?

<i>Datawords</i>	<i>Codewords</i>
00	000
01	011
10	101
11	110

Error detection using block coding

- The sender sends "011".
 - "011" received → receiver thinks dataword is "01".
 - "111" received → receiver knows an error has occurred.
 - "000" received → receiver thinks dataword is "00" (wrong!)
- This codeword set can detect 1-bit error, but cannot detect 2-bit error.
- But why? What determines how many errors we can detect?

<i>Datawords</i>	<i>Codewords</i>
00	000
01	011
10	101
11	110

Error detection using block coding

- Another block coding scheme
- What is k , n , and r ?

<i>Dataword</i>	<i>Codeword</i>
00	00000
01	01011
10	10101
11	11110

Error detection using block coding

- The sender sends codeword “01011”.
- The receiver receives “01001” (1-bit error).
- The receiver knows that there was an error.
- Can the receiver do more than that?

<i>Dataword</i>	<i>Codeword</i>
00	00000
01	01011
10	10101
11	11110

Error detection using block coding

- The sender sends codeword “01011”.
- The receiver receives “01001” (1-bit error).
- Among the codewords, 01011 is “the closest” codeword to 01001.
- Thus, the receiver guesses that the dataword was 01.

<i>Dataword</i>	<i>Codeword</i>
00	00000
01	01011
10	10101
11	11110

Error detection using block coding

- The sender sends codeword “01011”.
- The receiver receives “01001” (1-bit error).
- Let’s assume that there can be only 1-bit errors.
 - no 2 or more bits of error
- Then, this codeword set can correct the error.

<i>Dataword</i>	<i>Codeword</i>
00	00000
01	01011
10	10101
11	11110

Error detection using block coding

- The sender sends codeword “01011”.
- The receiver receives “00001” (2-bit error).
- In this case, the closest codeword is “00000” (wrong)
 - This codeword cannot correct 2-bit error.
- However, the receiver knows that there is an error.
 - Because there is no codeword “00001”.
 - This codeword set can detect 2-bit errors.
 - How about 3-bit errors?

<i>Dataword</i>	<i>Codeword</i>
00	00000
01	01011
10	10101
11	11110

Error detection using block coding

- What is the logic behind all this?

Hamming distance

- Number of different bits between two codewords
- $d(000, 000) = 0$
- $d(000, 011) = 2$
- $d(0100, 0010) = 2$
- $d(10101, 11110) = 3$

Minimum Hamming distance

- Given a codeword set, its MHD is the minimum of hamming distance between any two codeword in the set
- What is the MHD of this codeword set?

<i>Datawords</i>	<i>Codewords</i>
00	000
01	011
10	101
11	110

Minimum Hamming distance

- What is the MHD of this codeword set?

<i>Dataword</i>	<i>Codeword</i>
00	00000
01	01011
10	10101
11	11110

Hamming distance and error

- C_T : transmitted codeword
- C_R : received codeword
- If $d(C_T, C_R) = n$, this means there was an n-bit error
- Example
 - transmitted: 000
 - received: 011
 - $d(000, 011) = 2$, thus 2-bit error

MHD and error detection

- To detect an s -bit error, MHD of the codeword set must be larger than s .
- $d_{\min} > s$

MHD and error detection

- To detect an s -bit error, MHD of the codeword set must be larger than s .
- $d_{\min} > s$
- Suppose MHD of a codeword is $s+1$.
- Pick any codeword, and change s bits or less.
- This should not be in the codeword set, because then the MHD must be less than $s+1$.

MHD and error detection

- The MHD of this codeword set is 2.

<i>Datawords</i>	<i>Codewords</i>
00	000
01	011
10	101
11	110

- It means this codeword set can detect up to 1-bit errors.

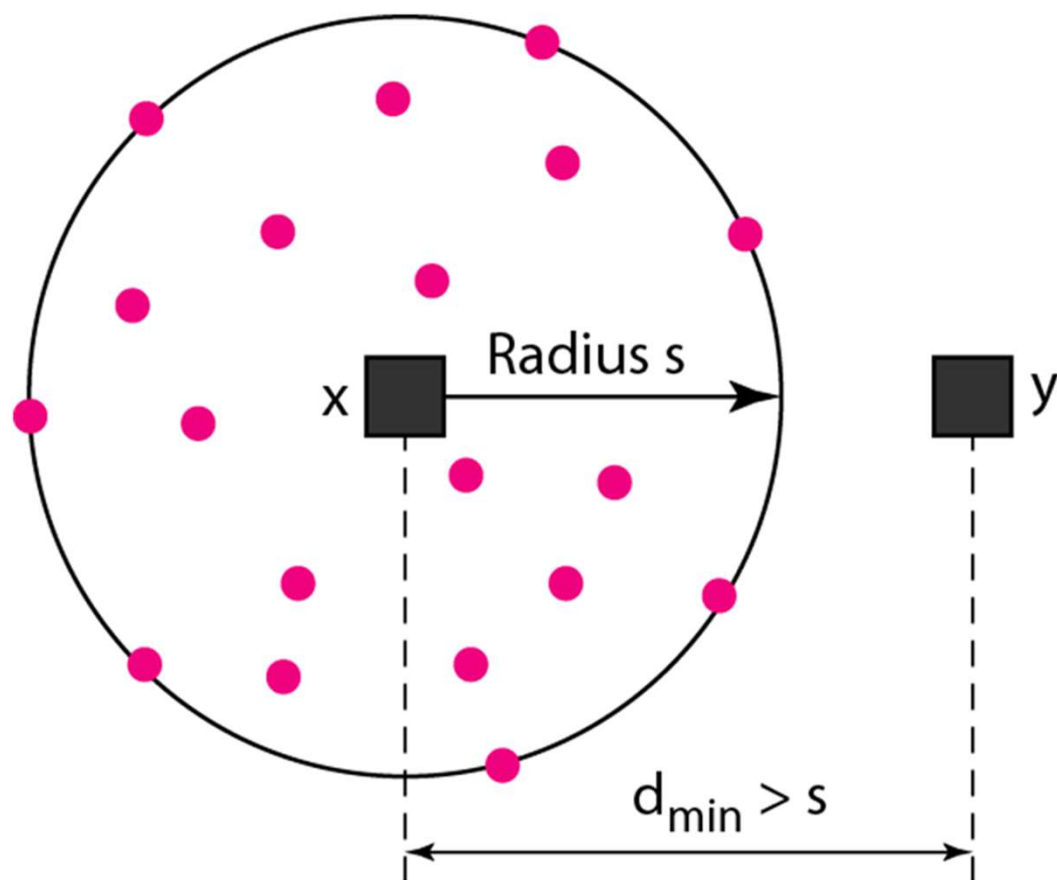
MHD and error detection

- The MHD of this codeword set is 3.

<i>Dataword</i>	<i>Codeword</i>
00	00000
01	01011
10	10101
11	11110

- It means this codeword set can detect up to 2-bit errors.

MHD and error detection



Legend



Any valid codeword



Any corrupted codeword
with 0 to s errors

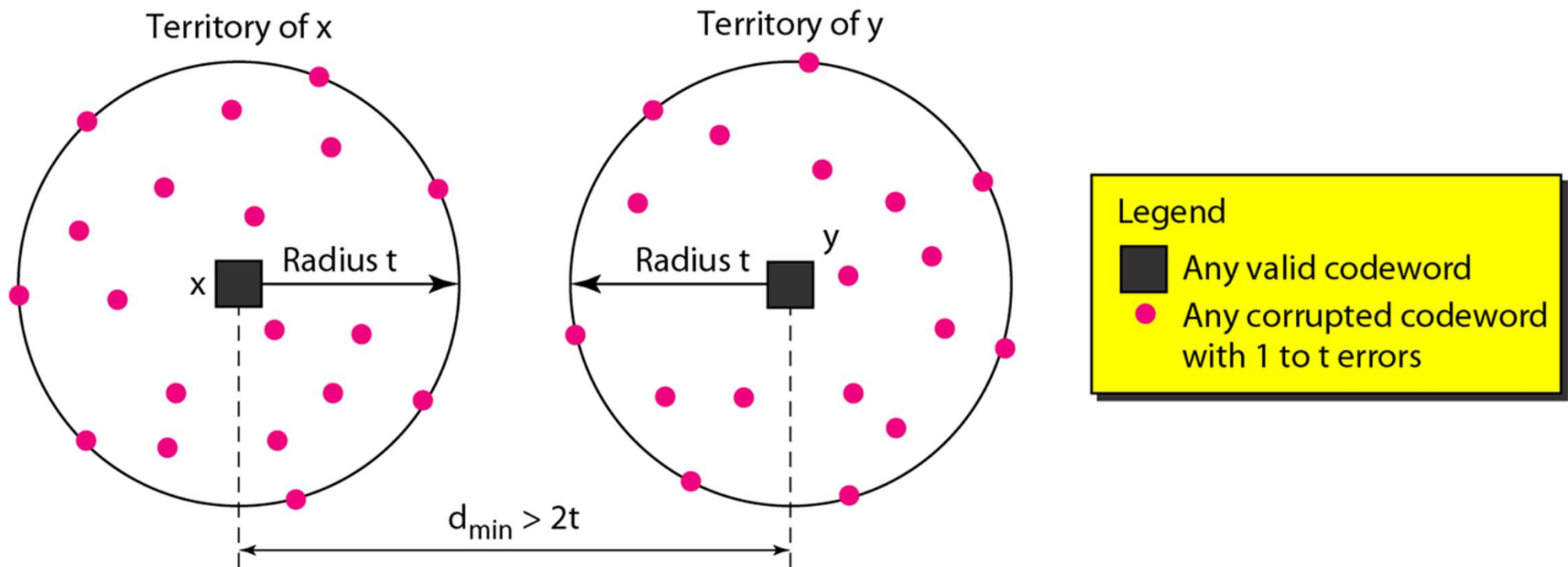
MHD and error correction

- To correct a t -bit error, MHD of the codeword set must be larger than $2t$.
- $d_{\min} > 2t$

MHD and error correction

- To correct a t -bit error, MHD of the codeword set must be larger than $2t$.
- $d_{\min} > 2t$
- Suppose MHD of a codeword is $2t+1$.
- Pick any codeword (C), and change t bits (C').
 - C' obviously is not in the codeword set.
- Choose the closest codeword to C' .
- It is C , since other codewords should have hamming distance of at least $t+1$.

MHD and error correction



MHD and error correction

- MHD of this codeword set is 3.

<i>Dataword</i>	<i>Codeword</i>
00	00000
01	01011
10	10101
11	11110

- This codeword set can detect up to 2-bit errors.
- This codeword set can correct up to 1-bit errors.

Example

- Suppose a codeword set has $MHD=4$.
- How many bit errors can this codeword set detect?
- How many bit errors can this codeword set correct?

Linear Block Codes

Linear block codes

- Pick any two codewords from the set
- XOR them
- The result codeword is in the codeword set
- If this is satisfied, this codeword set is called linear block codes

Linear block codes

- Is this linear block codes?

<i>Datawords</i>	<i>Codewords</i>
00	000
01	011
10	101
11	110

Linear block codes

- Is this linear block codes?

<i>Dataword</i>	<i>Codeword</i>
00	00000
01	01011
10	10101
11	11110

MHD of linear block codes

- Exclude codeword with all zeros
 - Count 1s for all other codewords
 - The minimum number of 1s is the MHD of the codeword set
-
- What is the MHD of this codeword set?

<i>Dataword</i>	<i>Codeword</i>
00	00000
01	01011
10	10101
11	11110

Simple Parity Check Codes

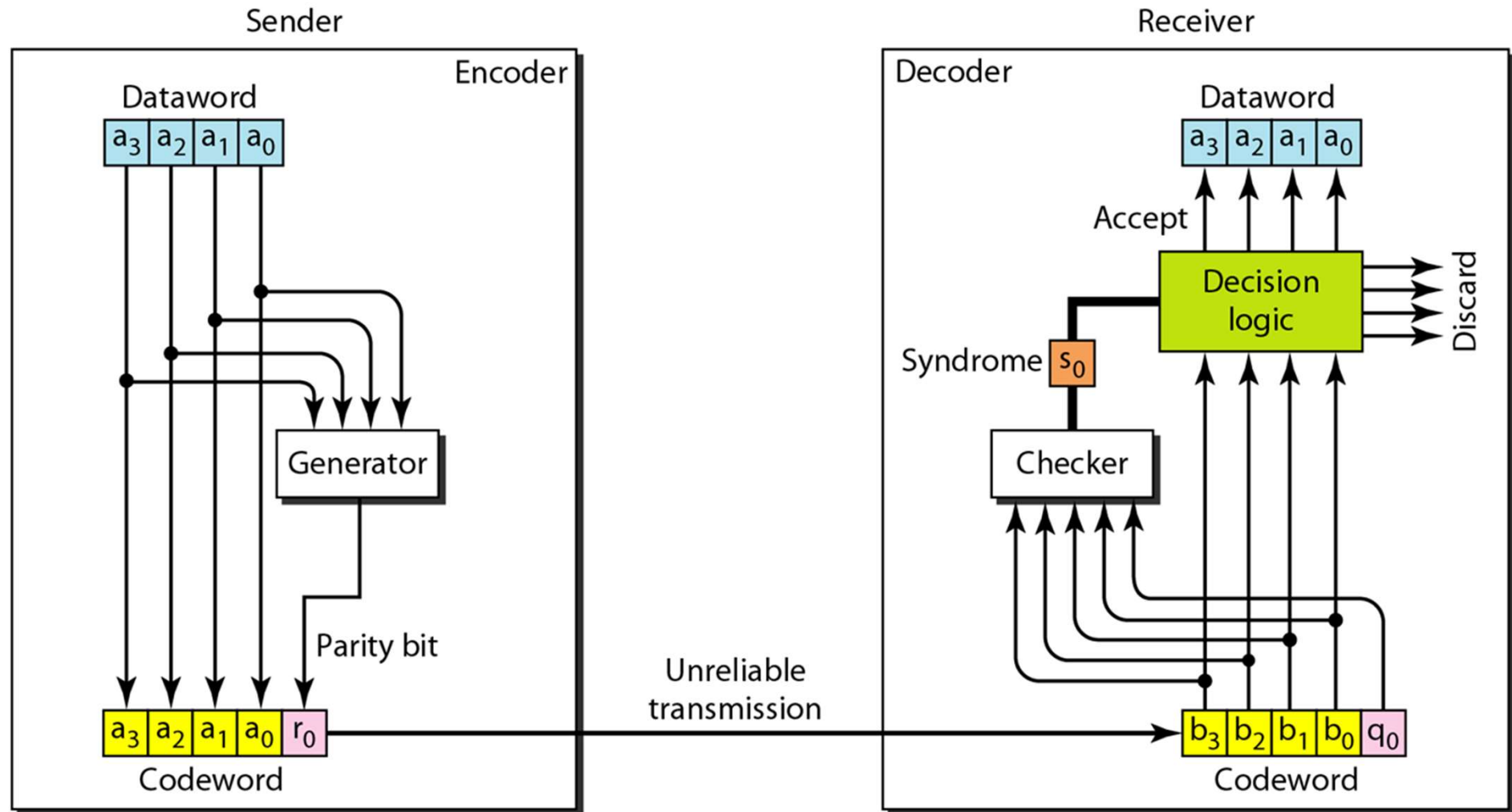
- k-bit dataword $\rightarrow$ (k+1)-bit codeword
 - Add 1 bit (parity bit)
- The parity bit should make the codeword have even number of 1s.

Simple Parity Check Codes

<i>Datawords</i>	<i>Codewords</i>	<i>Datawords</i>	<i>Codewords</i>
0000	00000	1000	10001
0001	00011	1001	10010
0010	00101	1010	10100
0011	00110	1011	10111
0100	01001	1100	11000
0101	01010	1101	11011
0110	01100	1110	11101
0111	01111	1111	11110

- $n = k + 1$
- $d_{\min} = 2$

Simple Parity Check Codes



Example

- Sender sends dataword '1011' using parity check codes.
 - Codeword is '10111'.
- Receiver receives '10111'. What happens?
- Receiver receives '10011'. What happens?
- Receiver receives '10110'. What happens?

Example

- Sender sends '10111'.
- Receiver receives '00110'. What happens?
- Receiver receives '01011'. What happens?

Performance of parity check codes

- Can parity check codes detect 1-bit errors?
- Can parity check codes detect 2-bit errors?
- Can parity check codes detect 3-bit errors?
- Can parity check codes detect 4-bit errors?

Performance of parity check codes

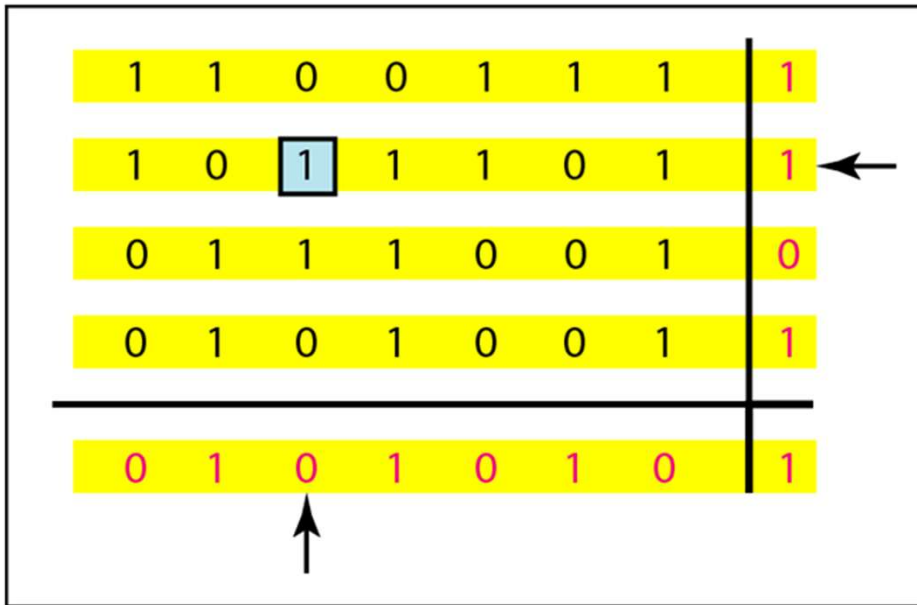
- Parity check codes can detect odd-bit errors.
- Pretty good!

2-dimensional parity

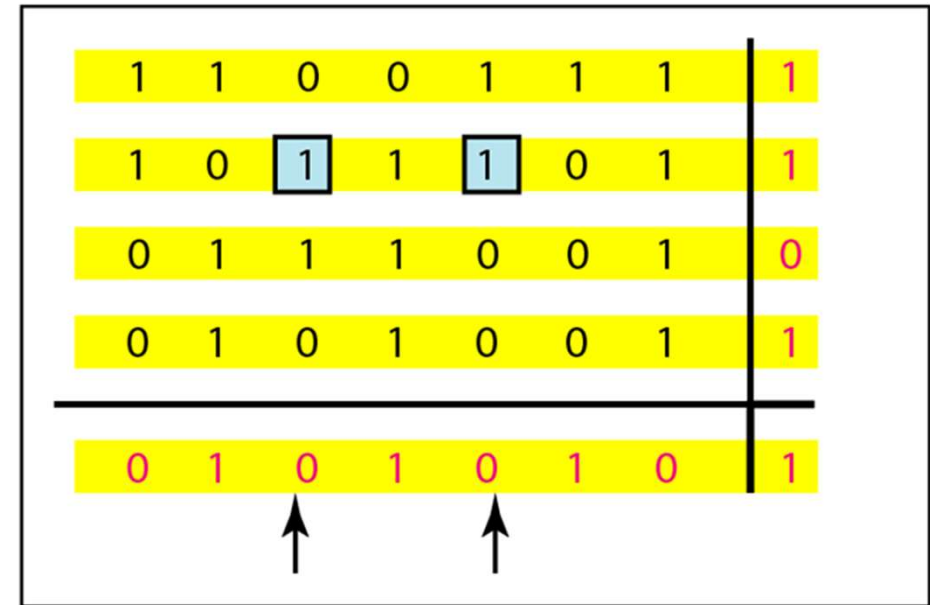
1	1	0	0	1	1	1	1	Row parities
1	0	1	1	1	0	1	1	
0	1	1	1	0	0	1	0	
0	1	0	1	0	0	1	1	
Column parities								
0	1	0	1	0	1	0	1	

a. Design of row and column parities

2-dimensional parity

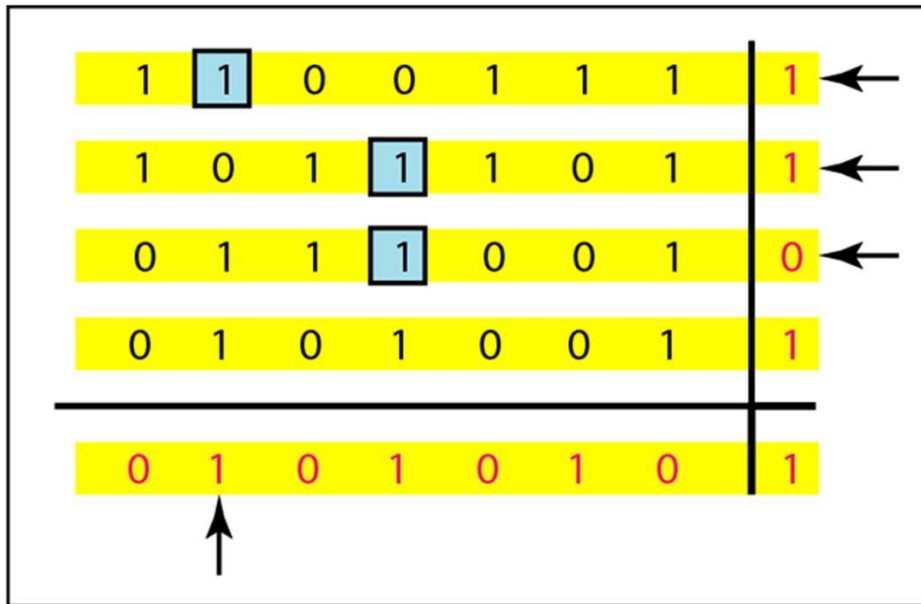


b. One error affects two parities

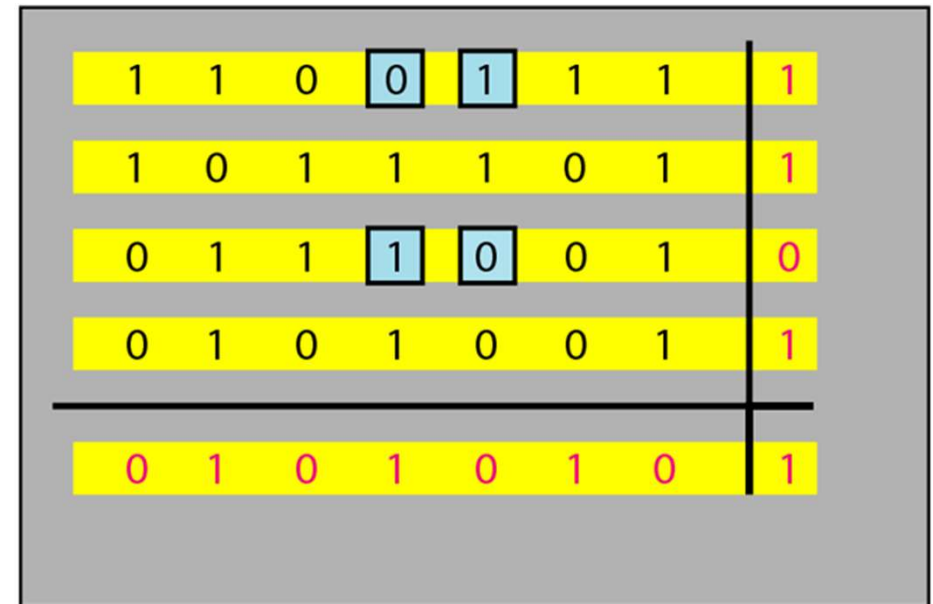


c. Two errors affect two parities

2-dimensional parity



d. Three errors affect four parities



e. Four errors cannot be detected

Performance of 2-D parity

- 1-bit errors: detect
 - 2-bit errors: detect
 - 3-bit errors: detect
 - 4-bit errors: cannot detect
-
- 2-D parity can detect some of 4-bit errors, but cannot detect all 4-bit errors.
 - If a codeword is said to detect n -bit errors, it should detect all of n -bit errors.
-
- 2-D parity can correct 1-bit error.

Hamming Code

- A family of error-correcting codes invented by Richard Hamming in 1950.
- $d_{\min} = 3$
 - detect up to 2-bit errors
 - correct up to 1-bit errors
- Hamming (n,k) code
 - convert a k-bit dataword into an n-bit codeword



Hamming (7,4) code

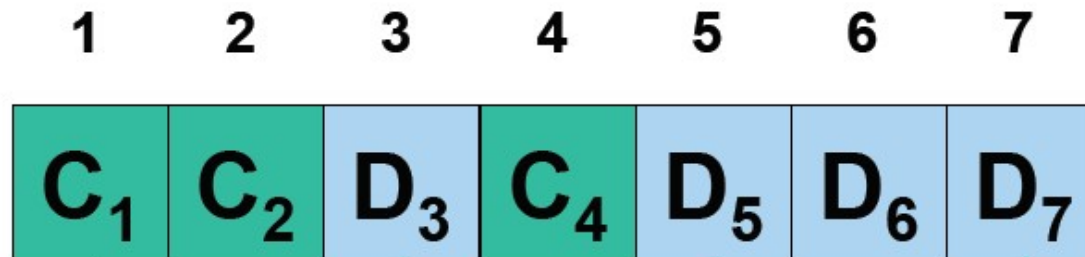
- Convert 4-bit datawords into 7-bit codewords

※ Note

- There are different ways to generate Hamming (7,4) code
- If you have textbooks, the coding method will be different from the slides
- I suggest that you should follow the slides (not the textbook)

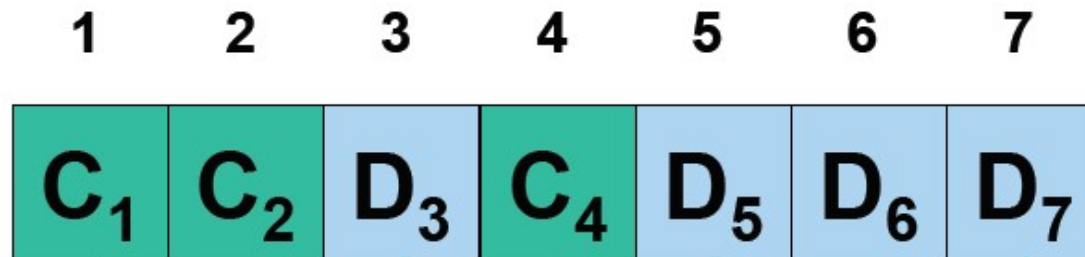
Hamming (7,4) code

- Number the bits from the left.
- If the index of the bit is a power of 2, it is a check bit.
 - 1st, 2nd, 4th bit
- If the index of the bit is not a power of 2, it is a data bit.
 - 3rd, 5th, 6th, 7th bit



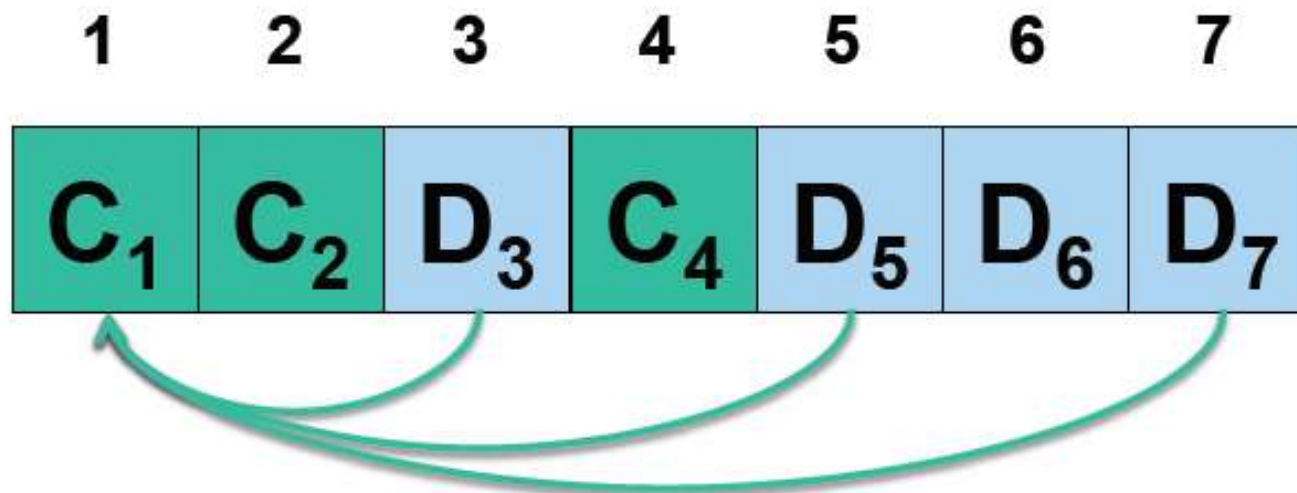
Hamming (7,4) code

- The 4 bits of the dataword go into the data bits
 - If dataword is “1011”
 - $D_3 = 1$
 - $D_5 = 0$
 - $D_6 = 1$
 - $D_7 = 1$



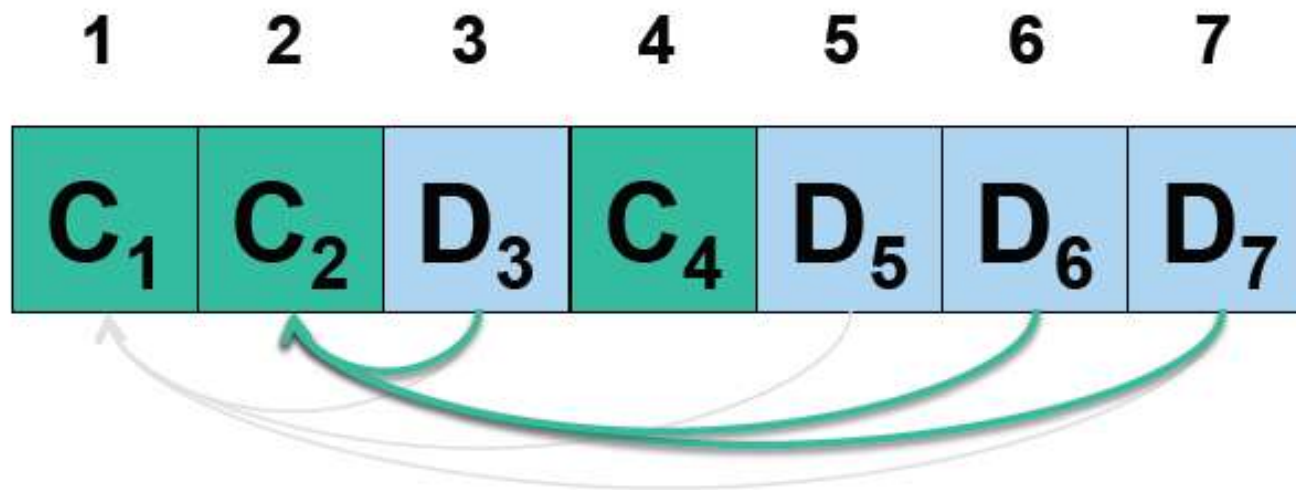
Hamming (7,4) code

- Check bits
 - j^{th} bit: select k^{th} bits where $j \& k = j$
 - $\&$: bitwise AND
- 1st bit: XOR values of 3rd, 5th, and 7th bits
 - $001 \& 011 = 001$, $001 \& 101 = 001$, $001 \& 111 = 001$



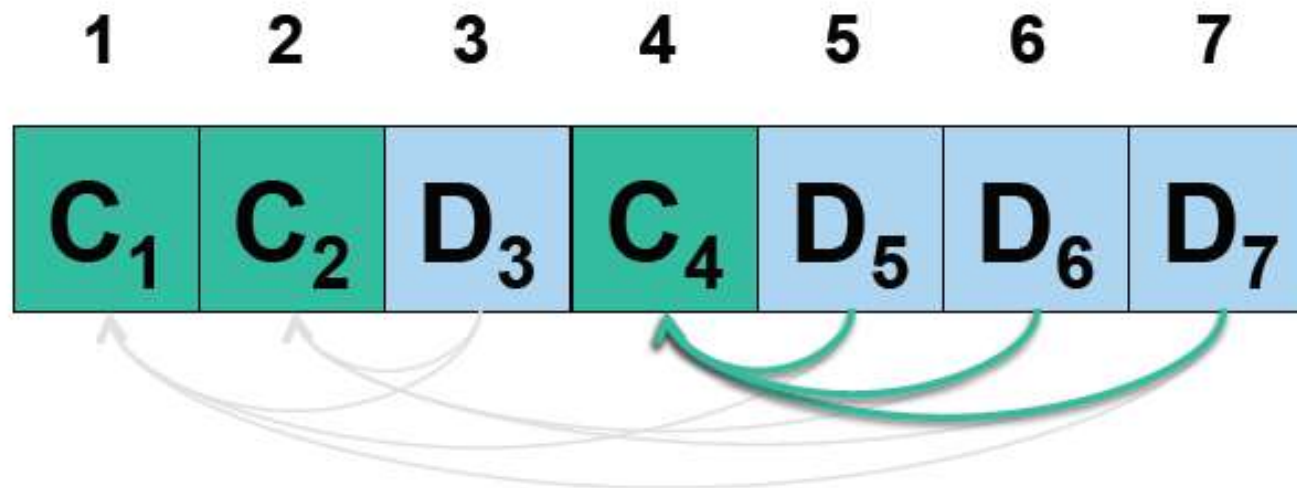
Hamming (7,4) code

- Check bits
 - j^{th} bit: select k^{th} bits where $j \& k = j$
 - $\&$: bitwise AND
- 2nd bit: XOR values of 3rd, 6th, and 7th bits

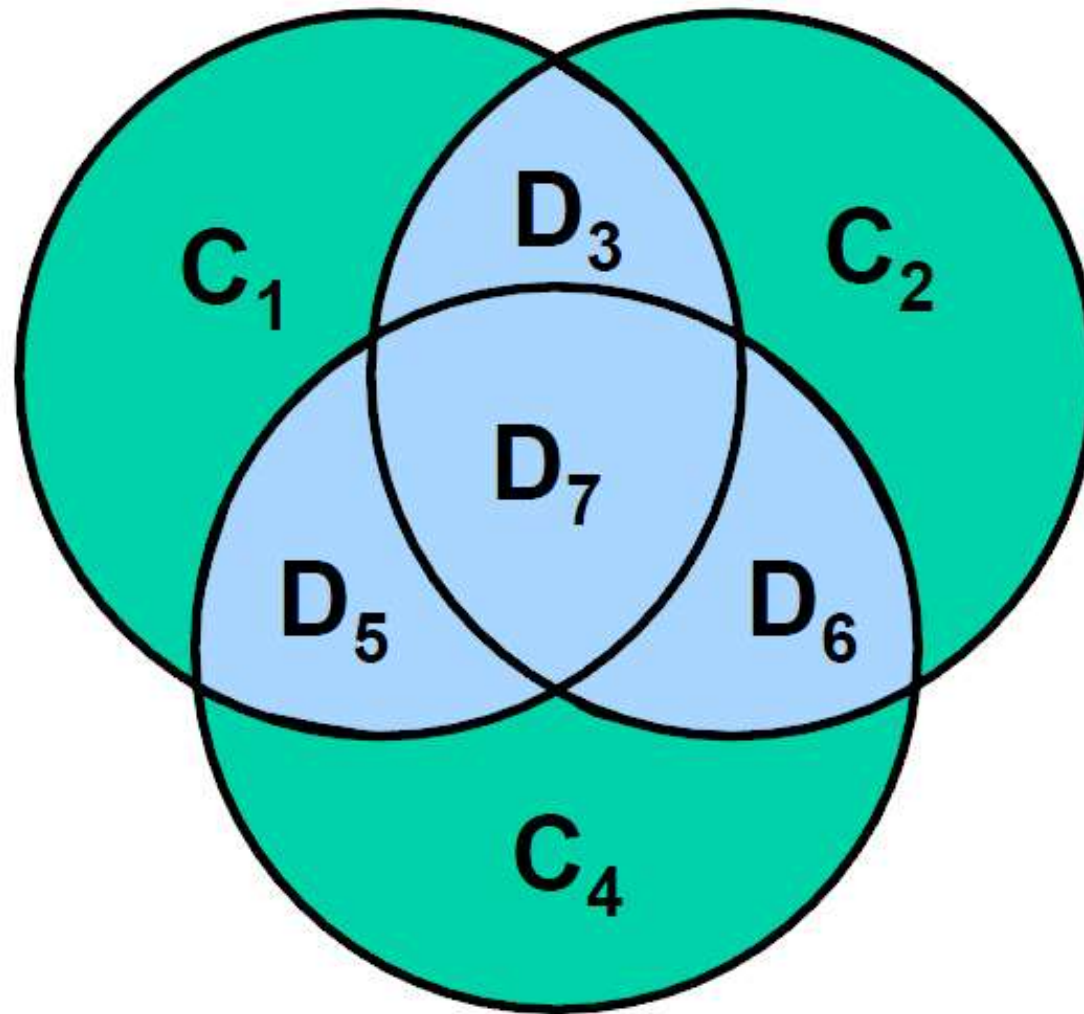


Hamming (7,4) code

- Check bits
 - j^{th} bit: select k^{th} bits where $j \& k = j$
 - $\&$: bitwise AND
- 4th bit: XOR values of 5rd, 6th, and 7th bits



Hamming (7,4) code



Hamming (7,4) code

- Exercise
- 0000 →
- 0011 →
- 1010 →
- 1110 →

Hamming (7,4) code

- What is the MHD of Hamming (7,4) code?
- Why?

Hamming (7,4) code

- Why (7,4)?
- Suppose we use n-bit codeword, where $n = 2^m - 1$ for a natural number m.
- Suppose the dataword is k bits, so that
- $k = n - \log_2(n+1)$
- Is there a better way to correct 1-bit errors?

Hamming (7,4) code

- To correct a 1-bit error in an n -bit codeword, the receiver should be able to correctly pick one of the following cases:
 - No error
 - 1st bit is error
 - 2nd bit is error
 - 3rd bit is error
 - ...
 - $(n-1)^{\text{th}}$ bit is error
 - n^{th} bit is error
- How many cases? $n+1$

Hamming (7,4) code

- For a n -bit codeword, $n+1$ pieces of information is needed in order to correct a 1-bit error.
- To embed $n+1$ pieces of information, we need $\log_2(n+1)$ bits
- Thus Hamming code uses minimum cost to correct 1-bit errors. There cannot be a better way.
 - Hamming code is described as a perfect code.

Voting (Repetition Code)

- The simplest way to increase hamming distance
- Convert '0' into N number of '0's
 - If $n = 3$, '0' $\rightarrow$ '000', '1' $\rightarrow$ '111'
- Makes hamming distance N
- If receiver receives '010', it is interpreted as 0 because the codeword has more 0s than 1s $\rightarrow$ voting

Voting (N=3) vs. Hamming (7,4)

- Efficiency of voting = $1/3 = 33.3\%$
- Efficiency of hamming (7,4) = $4/7 = 57.1\%$
- Both correct 1-bit errors, but Hamming code is more efficient

Hamming Codes

- Recall that to correct 1-bit error in 7-bit codeword, we need 3 control bits
 - 8 pieces of information
- In general, suppose we have a (2^k-1) bit codeword. Then, to correct 1-bit error we need k control bits
- If $k=4$, we obtain Hamming (15, 11) code
- If $k=5$, we obtain Hamming (31, 26) code
- The efficiency rate increases with k

Hamming (15,11) Code

- Dataword is 10110001011
- Convert dataword into codeword using hamming (15,11) code
- What is the efficiency rate?
 - $11/15 = 73.3\%$

Hamming Codes: Summary

- Size of the control bits = k
- Size of the codeword = $2^k - 1$
- Size of the dataword = $2^k - 1 - k$
- Efficiency rate = $(2^k - 1 - k) / (2^k - 1)$
- As k increases, efficiency rate increases
- However, as the dataword becomes longer, the possibility of having 2 or more bit-errors increases
→ detection rate decreases
- Hamming code only corrects 1-bit errors → practical systems use additional methods (e.g., CRC)

Checksum

Checksum

- An error detection method slightly more complex than parity check codes
- Used in IP, TCP, UDP headers for error detection

Checksum

- Suppose we want to send five 4-bit numbers
 - 7, 11, 12, 0, 6
- Add all numbers and place the sum as the last number
 - 7, 11, 12, 0, 6, 36
- The receiver checks for errors by adding all numbers and comparing with the last number
 - $7 + 11 + 12 + 0 + 6 = 36 \rightarrow$ no error!

Checksum

- We can also do this like the following.
- Add minus sign to the sum
 - 7, 11, 12, 0, 6, -36
- The receiver simply adds all numbers and see if it is zero
 - $7 + 11 + 12 + 0 + 6 + (-36) = 0 \rightarrow$ no error!

Wrapped Sum

- The problem is: 36 is not a 4-bit number
- We calculate a "wrapped sum"
 - 36 is "100100" in binary representation
 - keep the lower 4 bits, and add the number to the higher 2 bits

$$\begin{array}{r} 0100 \\ + \quad 10 \\ \hline 0110 \end{array}$$

- The wrapped sum of 36 is 6

Wrapped Sum

- What is the 4-bit wrapped sum of 45?
- $45 \rightarrow 101101$ (binary)

$$\begin{array}{r} 1101 \\ + \quad 10 \\ \hline 1111 \end{array}$$

- 4-bit wrapped sum of 45: 15

Wrapped Sum

- What is the 4-bit wrapped sum of 63?
- $63 \rightarrow 111111$ (binary)

$$\begin{array}{r} 1111 \\ + \quad 11 \\ \hline 10010 \end{array}$$

- This is still over 4 bits!

Wrapped Sum

- In this case, wrap one more time

$$\begin{array}{r} 0010 \\ + \quad 1 \\ \hline 0011 \end{array}$$

- 4-bit wrapped sum of 63: 3

Checksum

- Now we need to make the number negative (-6) using 4 bits
- Use 1's complement to represent -6
- 1's complement: toggle all bits
 - $6 = 0110$
 - $-6 = 1001$
- '1001' is 9 in decimal representation

Checksum

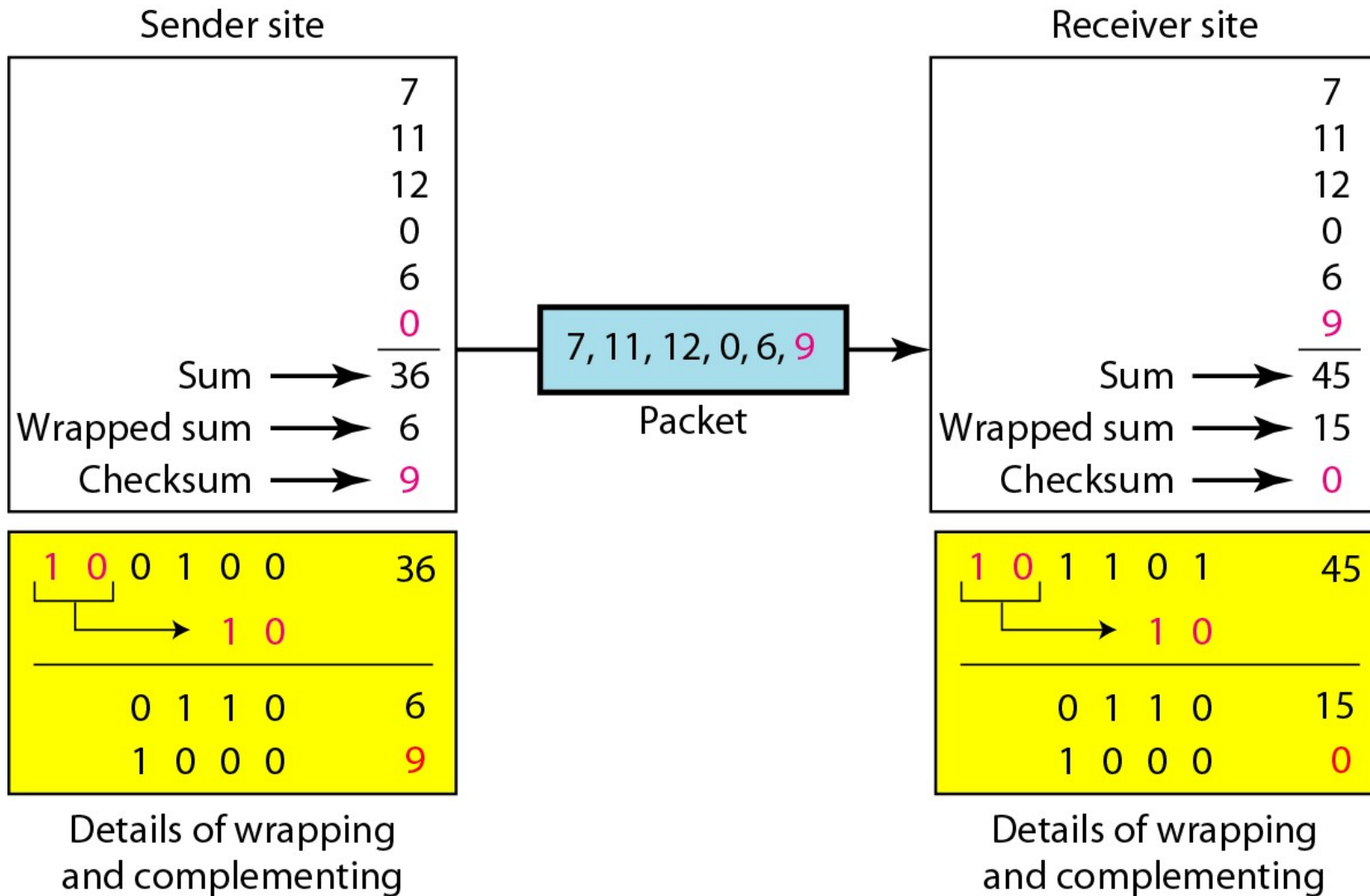
- The sender place '9' as the last number
 - 7, 11, 12, 0, 6, 9
- The receiver receives the numbers and adds them
 - $7 + 11 + 12 + 0 + 6 + 9 = 45$

- Calculate a wrapped sum of 45 (101101)

$$\begin{array}{r} 1101 \\ + \quad 10 \\ \hline 1111 \end{array}$$

- Find 1's complement of 1111 = 0000 → correct!
 - If the final value is not 0, then there is an error

Checksum



Cyclic Redundancy Check (CRC)

Cyclic Redundancy Check

- A powerful error detection method, stronger than checksum
- Complexity and power of CRC can be controlled by selecting "divisor (generator)"
- Used in Ethernet (CRC-32), Wi-Fi, USB, and many other protocols

Modulo Arithmetic

- A basic arithmetic used in CRC and other coding methods
- Modulo-N: use N numbers [0, N-1]
- Example: modulo-5
 - 0, 1, 2, 3, 4, 0, 1, 2, 3, 4, 0, 1, 2, 3, 4, 0, 1, 2, 3, 4
 - $3 + 2 = 0$
 - $4 + 3 = 2$
 - $2 - 3 = 4$

Modulo-2

- Use only 0 and 1
- $0 + 0 = 0$, $0 + 1 = 1$, $1 + 0 = 1$, $1 + 1 = 0$
- $0 - 0 = 0$, $0 - 1 = 1$, $1 - 0 = 1$, $1 - 1 = 0$
- Addition and subtraction lead to the same results
 - addition = subtraction

Modulo-2

- Addition and subtraction are equal to XOR

$$0 \oplus 0 = 0$$

$$1 \oplus 1 = 0$$

a. Two bits are the same, the result is 0.

$$0 \oplus 1 = 1$$

$$1 \oplus 0 = 1$$

b. Two bits are different, the result is 1.

$$\begin{array}{rcccccc} & 1 & 0 & 1 & 1 & 0 \\ \oplus & 1 & 1 & 1 & 0 & 0 \\ \hline & 0 & 1 & 0 & 1 & 0 \end{array}$$

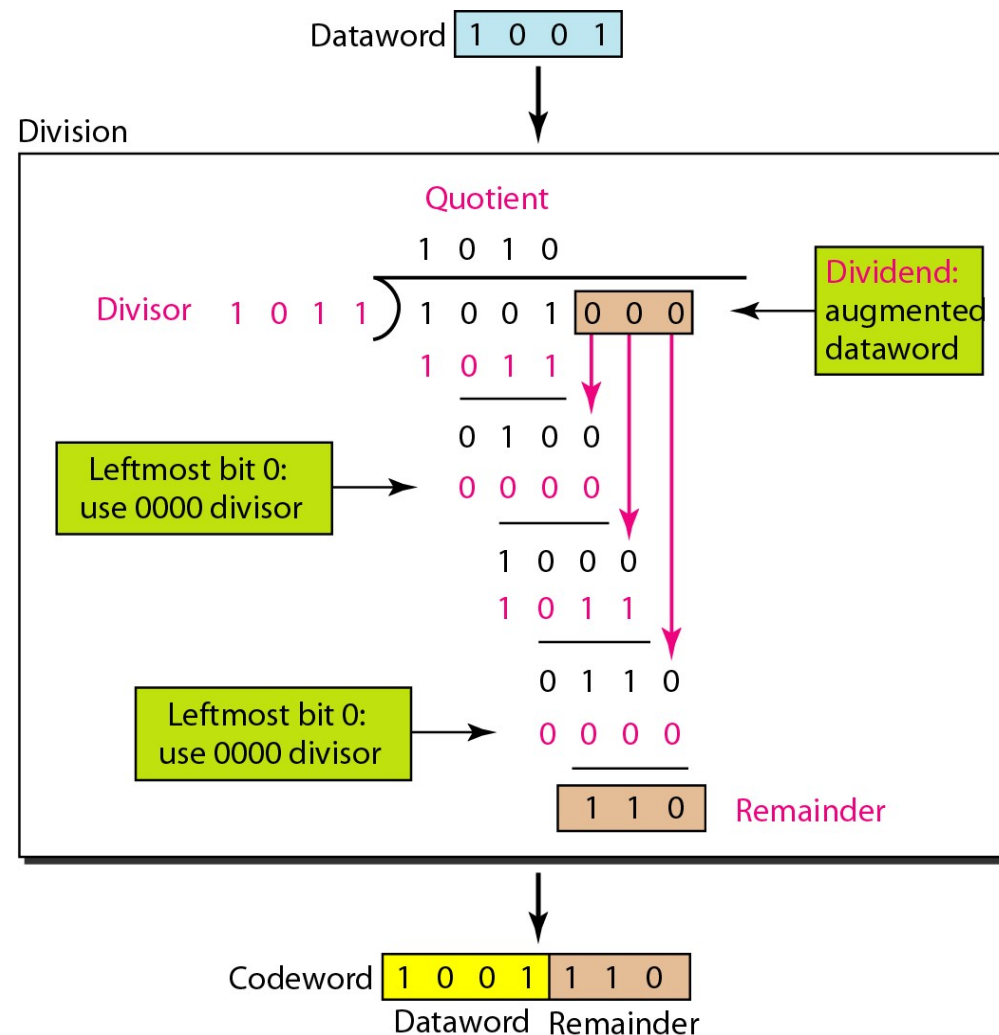
c. Result of XORing two patterns

Cyclic Redundancy Check

- Suppose we use the divisor '1011'
- We want to send the dataword '1001'
- Since the divisor is 4 bits, we add '000' after the dataword → '1001000'
- Divide '1001000' by the divisor '1011'
 - Use modulo-2 arithmetic

Cyclic Redundancy Check

- Dividing 1001000 by 1001 using modulo-2 arithmetic

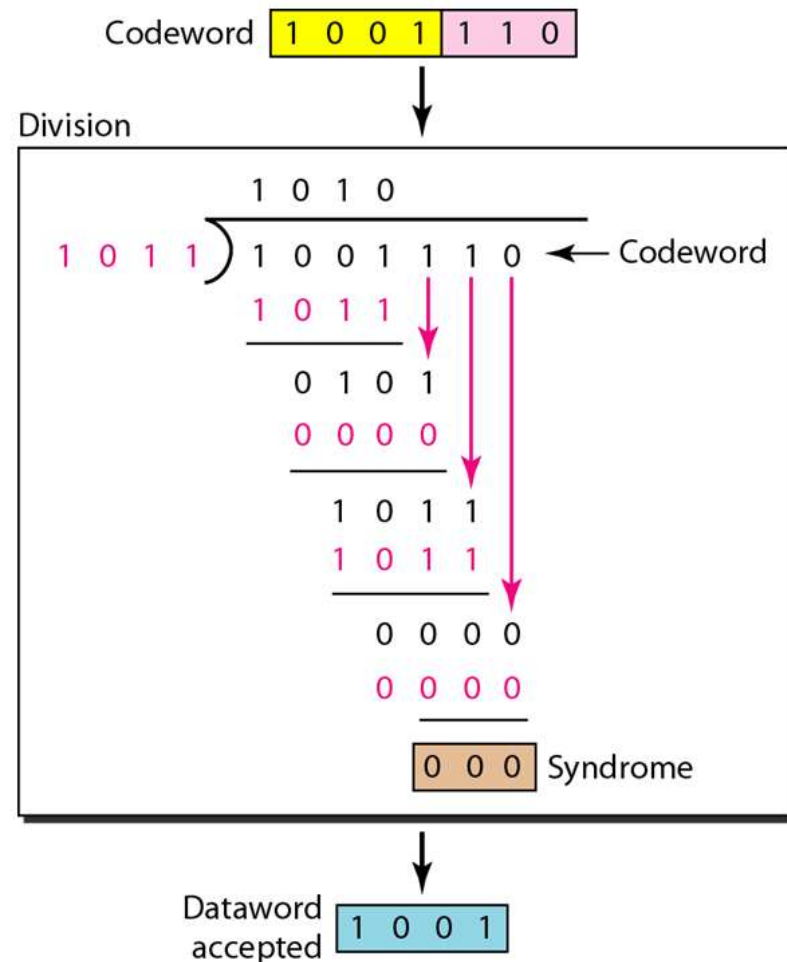


Cyclic Redundancy Check

- Since the remainder is '110', we add this to the dataword to make the codeword
→ 1001110
- Send this codeword

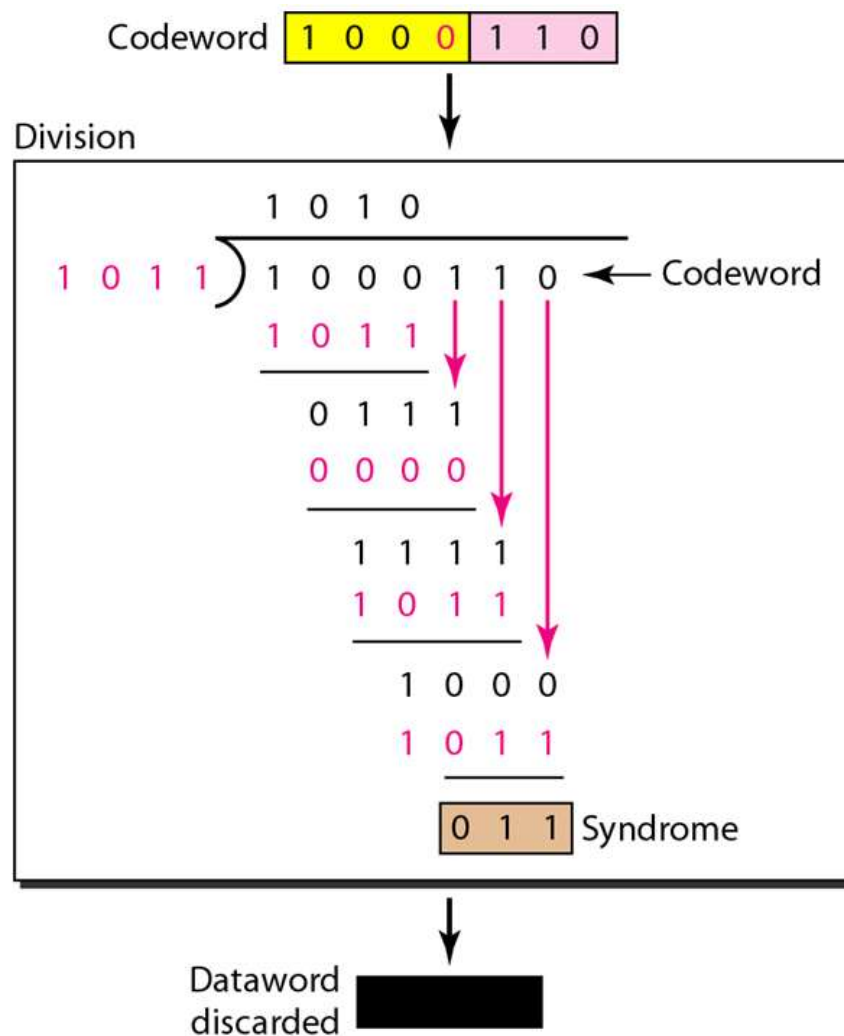
Cyclic Redundancy Check

- The receiver receives the codeword and divide the number by the divisor
 - If remainder is 0, then there is no error



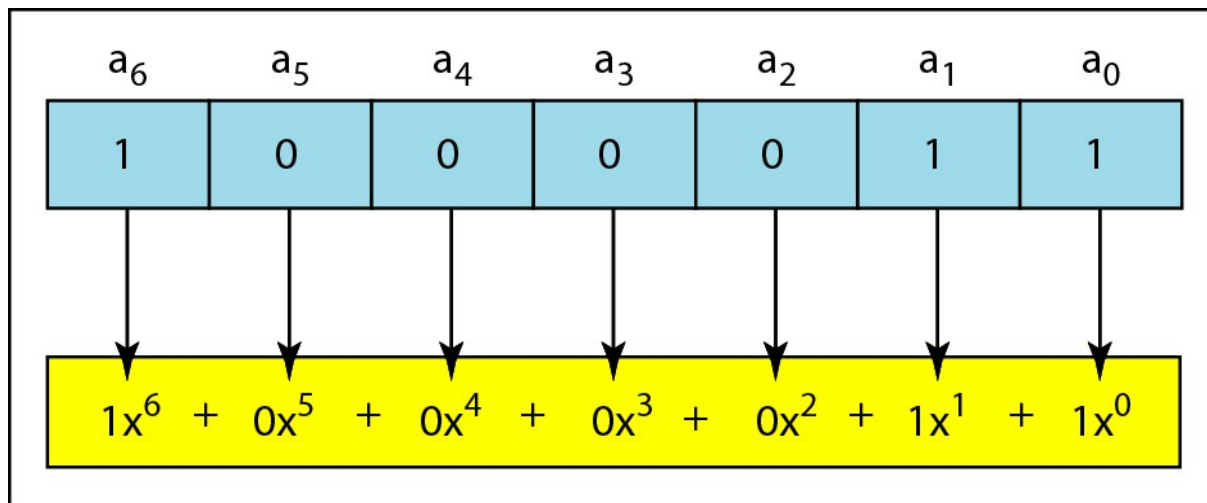
Cyclic Redundancy Check

- If the remainder is not 0, then error detected

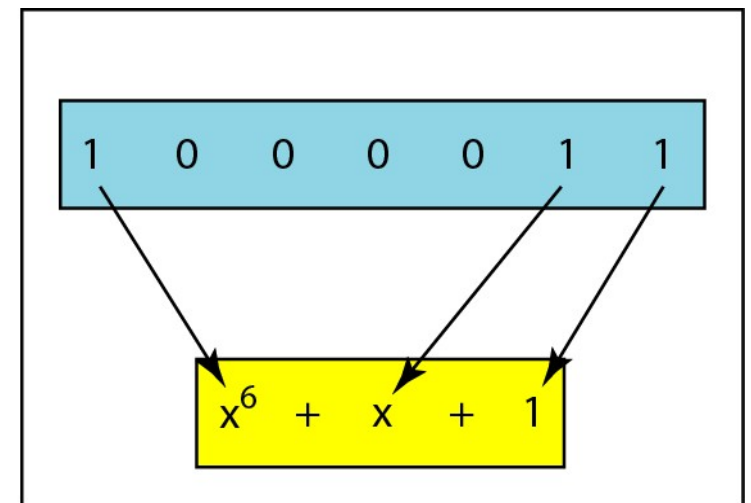


Binary Polynomials

- We can represent a binary number as a binary polynomial



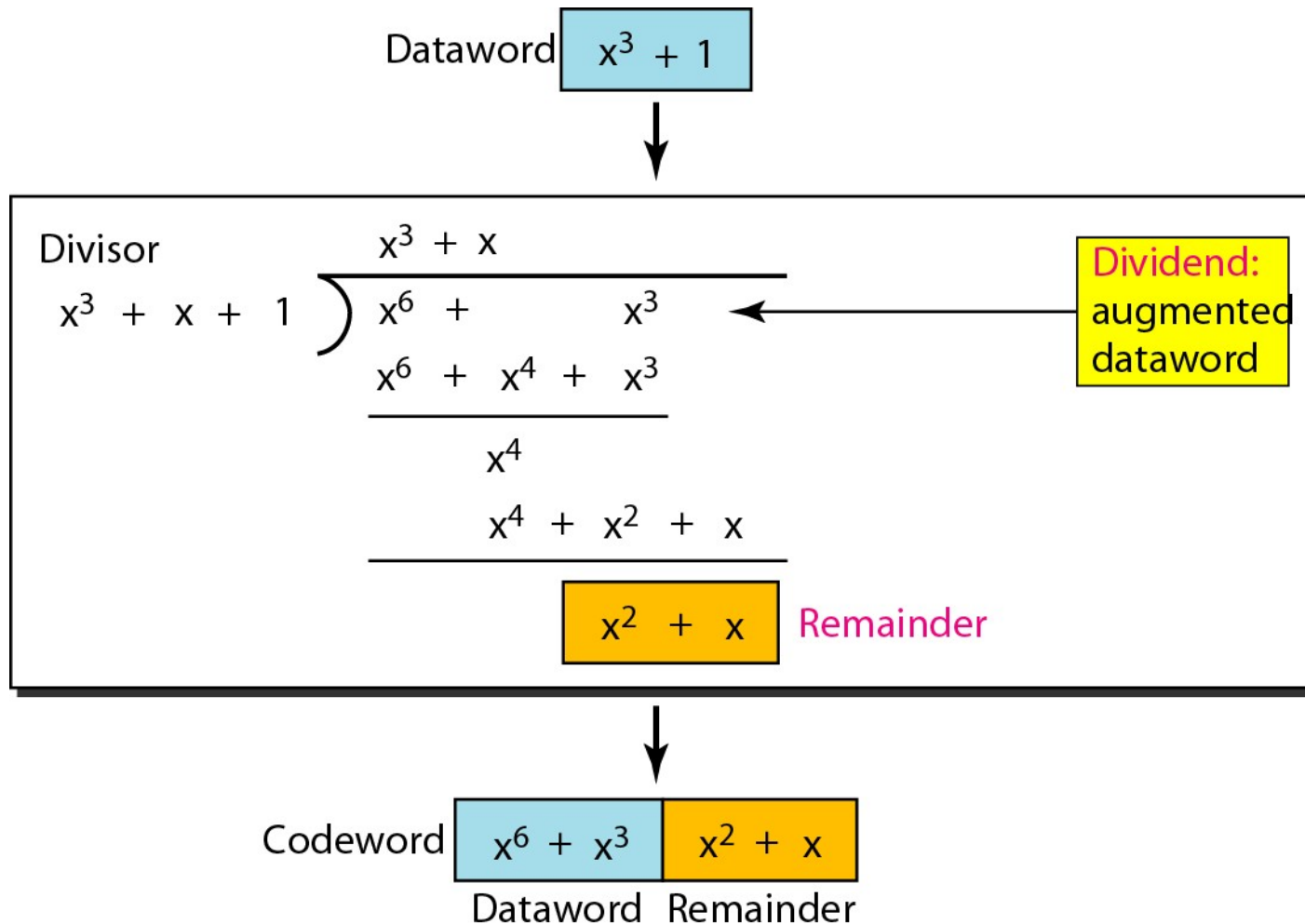
a. Binary pattern and polynomial



b. Short form

Binary Polynomials

- Division of binary polynomials



Cyclic Redundancy Check

- dataword(message): $M(x)$
- divisor: $g(x)$
- codeword: $T(x)$
- $T(x)$ is divisible by $g(x)$
 - Remainder of $T(x)/g(x)$ is 0

Cyclic Redundancy Check

- Sender sends $T(x)$
- Receiver receives $T'(x)$
- Receiver calculates $T'(x) / g(x)$
- If the remainder is zero, the receiver presumes $T(x) = T'(x)$ (no error)

Cyclic Redundancy Check

- When does CRC fail to detect an error?
- An error has occurred but, still the modified codeword $T'(x)$ is divisible by $g(x)$
→ error detection fails
- $T'(x) = T(x) + e(x)$
 - $e(x)$ is the error
 - If $T(x) + e(x)$ is divisible by $g(x)$, then the receiver thinks the codeword is correctly received
- Error detection performance of CRC depends on $g(x)$

Performance of CRC

- Suppose the divisor has x^0 term, and has at least one other term
 - e.g. $g(x) = x^2 + 1$
- Then, all 1-bit errors can be detected
- Why?

Performance of CRC

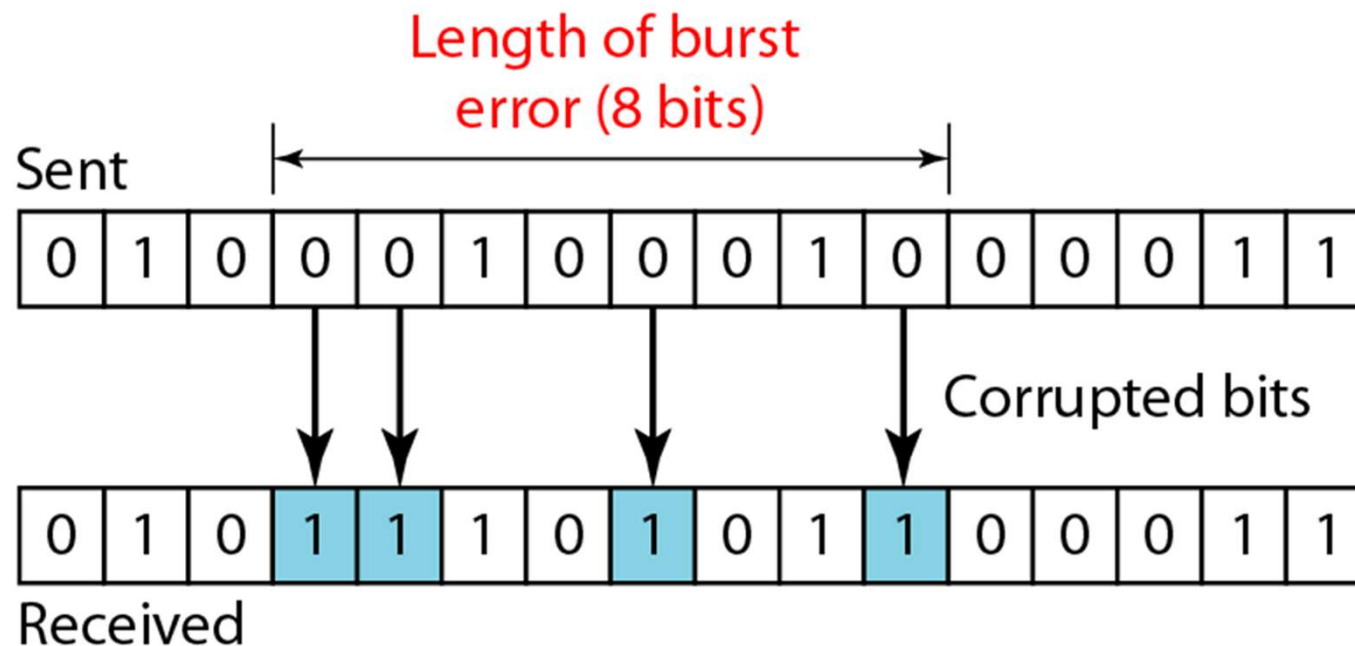
- Which of the following divisors can detect all 1-bit errors?
 - a. $x + 1$
 - b. x^3
 - c. 1

Performance of CRC

- If degree of the polynomial is n
 - detects all burst errors of size n or smaller
 - If size of the burst is $n+1$, the error can be detected with probability $1 - (1/2)^{n-1}$
 - If size of the burst is larger than $n+1$, the error can be detected with probability $1 - (1/2)^n$

Burst Error

- Multiple bits are corrupted
- Length of the burst error: from the first corrupted bit to the last corrupted bit



Performance of CRC

- Suppose the divisor is $x^6 + 1$
 - All burst errors with size 6 or less are detected
 - If the length of the burst is 7, then error detection probability is $31/32$
 - If the length of the burst is 8 or higher, then error detection probability is $63/64$
- Powerful!

Performance of CRC

- CRC divisors used in practice
 - CRC-8 is used by Bluetooth
 - CRC-16 is used by USB
 - CRC-32 is used by Ethernet, Wi-Fi
 - also used in ZIP (compression)

<i>Name</i>	<i>Polynomial</i>	<i>Application</i>
CRC-8	$x^8 + x^2 + x + 1$	ATM header
CRC-10	$x^{10} + x^9 + x^5 + x^4 + x^2 + 1$	ATM AAL
CRC-16	$x^{16} + x^{12} + x^5 + 1$	HDLC
CRC-32	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$	LANs

Use of CRC in Ethernet

- Ethernet Frame

Preamble	Dest MAC	Src MAC	Type	Payload	FCS
8 bytes	6 bytes	6 bytes	2 bytes	46–1500B	4 bytes

- Sender

- Compute CRC-32 over Dest MAC - Payload, place 32-bit remainder in FCS

- Receiver

- divide Dest MAC - FCS by CRC-32 polynomial
- Remainder = 0 → accept, otherwise → discard
- No retransmission; handled by upper layers (e.g., TCP)

- Performance

- $d_{\min} \geq 4$: detects all 1, 2, 3-bit errors
- Detects all burst errors of length ≤ 32
- CRC is computed in hardware (Network Interface Card) → very fast

End of Class
